

Applied Deep Learning | Hochschule Hof

Prof. Dr. Lars Ackermann-Igl

EFFICIENT LEARNING

Complete Exam-Ready Study Guide

- ◆ Optimization Dynamics (LR Schedulers + Normalization)
 - ◆ Generalization Strategies (Regularization + Dropout)
 - ◆ Hardware Acceleration (CUDA + TPUs)
- ◆ Representation Efficiency (Distillation + Quantization)
 - ◆ Exam Questions with Full Model Answers

01 | LR
Schedulers

02 | Normaliz
ation

03 | Regulari
zation

04 |
Hardware

05 |
Distillation

06 |
Quantization

07 | Exam
Q&A;

SECTION 1 — LEARNING RATE SCHEDULERS

Why Dynamic Learning Rates?

Until now we assumed the learning rate (LR) stays **constant** throughout training. However, a fixed LR is suboptimal: too large a rate causes overshooting and instability; too small a rate leads to very slow convergence or getting trapped in local minima.

Learning rate schedulers dynamically adjust the LR during training. Their goals are: (1) improve convergence speed, (2) escape local minima, and (3) ensure better generalisation.

The Four Scheduler Types

Step Decay	Reduces the learning rate by a fixed multiplicative factor every n epochs. Example: start at $LR=0.01$, multiply by 0.5 every 10 epochs $\rightarrow 0.01, 0.005, 0.0025, \dots$ Simple and predictable, but the sudden drops can be too abrupt.
Cosine Annealing	Smoothly decays the LR following a cosine curve from a maximum (η_{max}) to a minimum (η_{min}) over T_{max} steps. The smooth decrease avoids abrupt jumps. Highly popular for large-scale training. With 'warm restarts', the LR periodically resets to the maximum, helping the model escape local minima.
Cyclic (CLR)	Oscillates the LR between a lower and upper bound in a triangular or sinusoidal pattern. Forces wider exploration of the loss landscape. The parameter T_0 sets the cycle length and η_{min} sets the minimum. Especially useful when training dynamics are unstable.
Warm-Up	Starts with a very small LR and gradually ramps it up to the target LR over the first few epochs (the 'warm-up' phase), then may decay. Prevents destabilisation of the network in the early training phase when weights are randomly initialised and gradients are large. Standard practice for Transformers and large models.

EXAM TIP: Know the name, shape (how LR changes over time), and purpose of each scheduler. Warm-up = ramp up first. Step = sudden drops. Cosine = smooth curve. Cyclic = oscillates.

SECTION 2 — NORMALIZATION

Why Normalisation is Necessary

Problem: Unevenly scaled features (e.g., one input ranges 0–1, another 0–10,000) create an elongated, canyon-shaped loss surface. Large-scale features dominate gradient updates, while small-scale features barely contribute. This causes the optimizer to zigzag inefficiently instead of moving smoothly toward the minimum.

Goal of Normalisation: Rescale activations so that all features contribute equally, resulting in a circular (isotropic) loss surface that the optimizer can navigate smoothly and quickly.

Batch Normalisation

Batch Normalisation (BN) is an additional layer inserted into the network architecture. It normalises the output of the previous layer **across the mini-batch**.

Batch Norm — Training Phase (4 Steps):

1. Compute mean & std dev	For each feature across the current mini-batch of M samples, compute the mean ($\mu = (1/M) * \text{sum of activations}$) and standard deviation ($\sigma = \sqrt{(1/M) * \text{sum of squared deviations from mean}}$).
2. Normalise	Subtract the mean and divide by std dev: $A_{\text{hat}} = (A - \mu) / \sigma$. This re-centres activations to have zero mean and unit variance.
3. Scale and Shift	Apply two LEARNABLE parameters: γ (scale) and β (shift). $\text{BN_output} = \gamma * A_{\text{hat}} + \beta$. This restores the network's ability to represent any distribution — if needed, it can undo the normalisation by learning appropriate γ and β .
4. Moving Averages	Track exponential moving averages of μ and σ across batches (μ_{mov} and σ_{mov}). Needed because during INFERENCE there may be only one sample, so batch statistics are unavailable — the stored moving averages are used instead.

Batch Norm — Key Formulas:

Normalise: $A_{\text{hat}} = (A - \mu) / \sigma$ **Scale+Shift:** $\text{BN} = \gamma * A_{\text{hat}} + \beta$

Placement: BN can be added BEFORE the activation function (more common in practice) or AFTER the activation function.

Learnable parameters: β (shifts the mean) and γ (scales the variance). These let the network recover any distribution it needs.

Inference: Uses stored moving averages (μ_{mov} , σ_{mov}) since a single sample has no batch to compute statistics from.

Layer Normalisation

Layer Normalisation (LN) normalises **across all features of a single sample**, rather than across the batch. This makes it independent of batch size.

Property	Batch Normalisation	Layer Normalisation
Normalises across	Mini-batch (all samples, one feature)	All features (one sample, all features)
Requires large batches	Yes — noisy with small batches	No — works with batch size = 1
Train vs. Inference	Different: uses moving avgs at inference	Same operations in both modes
Best for	CNNs, fully connected layers, computer vision	RNNs, Transformers, NLP
Handles covariate shift	Across batch dimension	Across feature dimension (internal)

EXAM TIP: The key exam distinction: Batch Norm normalises ACROSS the batch for each feature; Layer Norm normalises ACROSS all features for each single sample. Batch Norm = good for CNNs. Layer Norm = good for Transformers/RNNs.

SECTION 3 — GENERALISATION STRATEGIES (REGULARISATION)

The Overfitting Problem

Problem: A flexible neural network can memorise the training data — learning noise and specific patterns that don't generalise to new data. This causes low training error but high test error (overfitting).

Core idea: Change the learning goal from just minimising training error to minimising BOTH training error AND model complexity: **Loss = Error(y, y_hat) + Complexity Penalty**

Approach 1: L1 Regularisation (Weight Penalty)

Principle: Add a penalty term to the loss function that grows with the absolute magnitude of the weights. This discourages large weights, forcing the model to use simpler, more robust representations.

Modified loss: $\text{Loss} = \text{Error}(y, y_{\text{hat}}) + \alpha * \sum(|w_i|)$

Effect on weight updates: Each weight is pushed toward zero by alpha at every step (if $w > 0$: subtract alpha; if $w < 0$: add alpha). This can drive some weights to exactly zero, effectively performing feature selection.

Alpha (scaling factor): Controls how strongly complexity is penalised. Small alpha = complexity is cheap (weak regularisation). Large alpha = complexity is expensive (strong regularisation, more weights pushed to 0).

Loss = Error(y, y_hat) + alpha * SUM |w_i| | dL/dw penalty = +alpha (w>0) or -alpha (w<0)

L2 Regularisation (also called Weight Decay) uses squared weights instead: $\alpha * \sum(w_i^2)$. Unlike L1, L2 does not push weights to exactly zero — it just keeps them small. L2 is smoother and preferred when all features are relevant.

Approach 2: Dropout

Principle: During training, randomly deactivate (drop) a fraction of neurons in a layer at each forward pass. Dropped neurons receive no gradient update during backpropagation (no weight update).

Why it works: By randomly removing neurons, the network cannot rely on any single neuron or co-adaptation of neurons. It is forced to learn redundant, distributed representations — many neurons learn to detect the same patterns independently. This is equivalent to training an ensemble of many different, smaller networks.

Dropout rate (p): The probability of dropping each neuron. Common values: $p = 0.2$ to 0.5 for hidden layers.

During inference: All neurons are active, but their outputs are scaled by $(1-p)$ to maintain the same expected output magnitude as during training.

Approach 3: Early Stopping

Principle: Monitor validation loss during training and stop training when the validation loss stops improving (or starts worsening), even if training loss continues to decrease.

Why it works: Overfitting occurs when the model has trained 'too long' and has started memorising training data. The validation loss curve increases after the 'early stopping epoch' (the point of best generalisation). Stopping there captures the model at peak generalisation performance.

Implementation: Use a 'patience' parameter — wait P epochs without validation improvement before stopping. Save the best model checkpoint during training.

Regularisation Methods — Quick Comparison

Method	How It Works	When to Use
L1 Regularisation	Adds $ w $ penalty to loss; pushes weights toward 0 or exactly 0	When you suspect many features are irrelevant (sparse solutions)
L2 Regularisation	Adds w^2 penalty; keeps all weights small but non-zero	Default regulariser; when all features may be relevant
Dropout	Randomly deactivates neurons during training each step	Deep networks; prevents co-adaptation; acts like ensembling
Early Stopping	Stops training when validation loss stops improving	Always useful; free regularisation with minimal overhead

EXAM TIP: Regularisation questions often ask: (1) What is the problem it solves? (Overfitting). (2) How does it change the loss? (Adds a penalty term). (3) What does dropout do at inference? (All neurons active, outputs scaled by $1-p$).

SECTION 4 — HARDWARE ACCELERATION (CUDA & TPUs)

The Compute Bottleneck

Training neural networks requires billions of floating-point matrix multiplications (forward + backward passes). A standard CPU processes these sequentially — far too slow for large models. Specialised hardware is needed to parallelise these operations.

CUDA — Compute Unified Device Architecture

What it is: CUDA is a parallel computing platform and API developed by NVIDIA. It allows programs to run thousands of operations simultaneously on GPU cores, rather than one at a time on CPU cores.

Why it matters for deep learning: Neural network training is dominated by matrix multiplications ($Y = W * X$). GPUs have thousands of cores optimised for exactly this type of SIMD (Single Instruction, Multiple Data) parallel processing — performing the same operation on many data elements simultaneously.

Integration: TensorFlow and PyTorch use CUDA automatically when a GPU is available. Keras (which runs on TensorFlow) calls CUDA/cuDNN under the hood — the developer does not need to write any CUDA code directly. Device can be explicitly set with `tf.device('/GPU:0')` or `'/CPU:0'`.

TPUs — Tensor Processing Units

What it is: A TPU is a custom-designed AI accelerator chip by Google, built specifically for neural network matrix operations — unlike GPUs which are general-purpose graphics chips adapted for deep learning.

Key strengths over GPUs:

- **High throughput for matrix ops:** TPUs are optimised specifically for large matrix multiplications — the backbone of neural networks.
- **Massive parallelism:** Perform the same operation on many data elements simultaneously (SIMD-style).
- **High Memory Bandwidth (HBM):** Feed data to compute units very quickly, reducing bottlenecks between memory and processing.
- **Energy efficiency:** More performance per watt than CPUs or GPUs.

Bottom line: TPUs accelerate training by performing the right operations in parallel, with high memory bandwidth and energy efficiency — enabling faster training, lower cost, and the ability to train larger models.

Hardware	Who Makes It	Key Strength	Framework Support
CPU	Intel / AMD	Sequential tasks, general computing	All frameworks
GPU + CUDA	NVIDIA	Parallel matrix ops; fast for DL training	PyTorch, TensorFlow, Keras
TPU	Google	Ultra-fast matrix ops; energy efficient; HBM	TensorFlow/Keras, JAX, PyTorch (XLA)

EXAM TIP: CUDA = NVIDIA's software platform for GPU parallel computing. TPU = Google's custom chip for matrix operations. Both exploit SIMD parallelism. TPUs have higher memory bandwidth (HBM) than GPUs.

SECTION 5 — REPRESENTATION EFFICIENCY: MODEL DISTILLATION

The Core Idea

Problem: Large models (e.g., GPT-4, ResNet-152) achieve high accuracy but require enormous compute resources for inference — impractical for mobile devices, edge systems, or real-time applications.

Model/Knowledge Distillation (also called teacher-student learning) transfers the knowledge from a large, powerful model (the **teacher**) into a smaller, efficient model (the **student/pupil**). The student learns to mimic the teacher's behaviour, achieving performance close to the teacher at a fraction of the computational cost.

How Distillation Works

Step 1	Train the Teacher Train a large, high-performing model (teacher) on the task. This model is computationally expensive but very accurate.
Step 2	Generate Soft Targets Run the training data through the teacher and collect its output logits (raw scores before softmax) or softened probability distributions. These 'soft labels' contain richer information than hard one-hot labels — e.g., 'this is 70% cat, 25% lynx, 5% dog' captures relationships between classes.
Step 3	Train the Student Train a small student model using both the original hard labels AND the teacher's soft targets. The distillation loss measures how well the student matches the teacher's output distribution.
Step 4	Deploy the Student At inference time, only the small student model is used — fast, memory-efficient, and close in accuracy to the much larger teacher.

Implementation Variants

Soft Target Distillation	Soft Target Distillation: Student trained to match teacher's output probability distribution (logits/soft targets) using KL divergence loss. The soft labels provide more signal than one-hot labels.
Cross-Modal Distillation	Cross-Modal Distillation: Teacher and student operate in different modalities — e.g., teacher works with images, student with text. Knowledge transferred across domains.
GAN-based Distillation	GAN-based Distillation: A discriminator is used to make the student's outputs indistinguishable from the teacher's, using adversarial training.

EXAM TIP: Key terms: Teacher = large model, Student/Pupil = small model. Soft labels = teacher's probability outputs (richer than hard 0/1 labels). Goal = compact, efficient student that approaches teacher accuracy.

SECTION 6 — REPRESENTATION EFFICIENCY: QUANTIZATION

The Core Problem

Neural network weights are typically stored as 32-bit floating point numbers (FP32 — 32 bits per weight, 7 decimal places of precision). For a model with billions of parameters, this requires enormous memory.

Key insight: Neural network weights rarely use the full FP32 value range (-3.4×10^{38} to $+3.4 \times 10^{38}$). In practice, weights cluster in a much smaller range. We can represent them with fewer bits without significant accuracy loss.

What Quantization Does

Quantization maps high-precision floating-point weights (FP32) to lower-precision integer representations (e.g., INT8 = 8-bit integer, range -128 to 127).

A **quantization function Q** maps the FP32 range $[f_min, f_max]$ to the integer range $[q_min, q_max]$. For INT8: $q_min = -128$, $q_max = 127$.

Memory saving: FP32 uses 4 bytes per weight. INT8 uses 1 byte per weight. That is a **4x memory reduction**! Smaller models fit in memory, load faster, and run faster on hardware optimised for integer arithmetic.

Two Types of Quantization:

Type	Description	Accuracy Trade-off	When Used
PTQ Post-Training Quantization	Quantize weights AFTER training is complete. Simple to apply to any pre-trained model.	Some accuracy loss; may degrade if model is sensitive to precision	Quick deployment; when re-training is not feasible
QAT Quantization n-Aware Training	Simulate quantization during training — model adapts to lower precision. Quantization is applied as a calibration step.	Better accuracy than PTQ; model explicitly learns to handle lower precision	When accuracy is critical; when PTQ accuracy drop is too large

EXAM TIP: FP32 (32 bits) → INT8 (8 bits) = 4x memory saving. PTQ = quantize after training (simple but some accuracy loss). QAT = simulate quantization during training (better accuracy, more expensive).

SECTION 7 — EXAM QUESTIONS WITH FULL MODEL ANSWERS

EXAM QUESTION Q1 [~3 Points]

What are learning rate schedulers and why are they used? Name and briefly describe two types of learning rate schedulers.

MODEL ANSWER

Definition: Learning rate schedulers dynamically adjust the learning rate during training rather than keeping it constant.

Why they are used: A fixed learning rate is suboptimal — too high causes overshooting and instability, too low causes slow convergence or entrapment in local minima. Schedulers aim to improve convergence speed, escape local minima, and ensure better generalisation.

Step Decay: Reduces the learning rate by a fixed multiplicative factor every n epochs. Simple and predictable — e.g., multiply by 0.5 every 10 epochs. Drawback: the sudden drops can be too abrupt.

Cosine Annealing: Smoothly decays the learning rate following a cosine curve from a maximum to a minimum value. With warm restarts, the LR periodically resets to maximum, helping escape local minima. Highly suitable for large-scale training.

EXAM QUESTION Q2 [~4 Points]

Why is normalisation necessary in neural network training? Explain the principle of Batch Normalisation and describe its two learnable parameters.

MODEL ANSWER

Why normalisation: Unevenly scaled features cause the loss surface to be elongated like a canyon. Large-scale features dominate gradient updates and small-scale features contribute little, causing inefficient, zigzagging optimisation. Normalisation reshapes the loss surface to be more circular, enabling smooth and efficient gradient descent.

Batch Normalisation principle: BN is an additional layer inserted into the network. During training, it computes the mean and standard deviation of each feature across the current mini-batch, then normalises the activations to have zero mean and unit variance: $\hat{A} = (A - \mu) / \sigma$.

Learnable parameter beta (shift): Shifts the mean of the normalised output. Allows the network to move the distribution to any desired centre point rather than being forced to always use zero mean.

Learnable parameter gamma (scale): Scales the variance of the normalised output. Allows the network to adjust the spread of the distribution. Together, beta and gamma let BN undo its own normalisation if the original distribution was actually optimal — making BN lossless in expressiveness.

EXAM QUESTION Q3 [~3 Points]

Compare Batch Normalisation and Layer Normalisation. When would you use each?

MODEL ANSWER

Batch Normalisation normalises across the mini-batch dimension: for each feature, it computes statistics (mean and std dev) across all samples in the batch. This makes BN dependent on batch size — it struggles with very small batches (noisy statistics). Works well for computer vision tasks (CNNs, fully connected layers) where features across different images are comparable.

Layer Normalisation normalises across all features for each individual sample independently. It computes statistics within one sample. This makes LN completely independent of batch size — it works identically at training and inference, even with a single sample. Works well for sequential models (RNNs, Transformers) where the batch dimension is less meaningful.

Use Batch Norm for CNNs and vision tasks with adequate batch sizes. **Use Layer Norm** for NLP, RNNs, Transformers, or when batch sizes are very small.

EXAM QUESTION Q4 [~3 Points]

What is regularisation in the context of neural networks? Explain how L1 regularisation works and what the scaling factor alpha controls.

MODEL ANSWER

Regularisation addresses overfitting — when a model memorises training data and performs poorly on new data. The core idea is to change the learning objective from minimising only training error to minimising training error PLUS a complexity penalty, discouraging the model from becoming too complex.

L1 regularisation: Adds a penalty term equal to alpha times the sum of the absolute values of all weights ($\alpha * \sum |w_i|$) to the loss function. This makes large weights expensive. During gradient descent, each weight is additionally pushed toward zero by alpha at every update step. This can drive some weights to exactly zero, effectively removing connections from the network (acting as feature selection).

Alpha (scaling factor): Controls the strength of regularisation. A small alpha means model complexity is cheap — weak regularisation. A large alpha means model complexity is expensive — strong regularisation, more weights are pushed toward zero. If alpha is too large, the model underfits.

EXAM QUESTION Q5 [~3 Points]

Explain the Dropout technique. How does it work during training, and how does behaviour differ during inference?

MODEL ANSWER

Dropout — during training: At each forward pass, each neuron in a dropout layer is independently and randomly deactivated with probability p (the dropout rate). Deactivated neurons output zero and receive no gradient update during backpropagation. A different random subset is dropped each batch, so the network never relies on any single neuron.

Why it prevents overfitting: By forcing the network to work with random subsets of neurons, it cannot co-adapt neurons to memorise specific training examples. The network must learn redundant, distributed representations. This is equivalent to training an ensemble of many different network architectures and averaging their predictions.

During inference: All neurons are active (no dropout is applied). However, the output of each neuron is scaled by $(1 - p)$ to compensate for the fact that during training only a fraction $(1-p)$ of neurons were active on average. This keeps the expected output magnitude consistent.

EXAM QUESTION Q6 [~3 Points]

What is model distillation (knowledge distillation)? Explain the teacher-student concept and describe the role of soft labels.

MODEL ANSWER

Model distillation is a technique for compressing a large, powerful model (the teacher) into a smaller, more efficient model (the student/pupil). The student learns to mimic the teacher's behaviour rather than learning directly from the raw training data alone.

Teacher: A large, highly accurate model — computationally expensive to run. Trained first on the full task.

Student/Pupil: A small, compact model — fast to run, low memory. Trained to match the teacher's outputs.

Role of soft labels: Instead of training the student with hard one-hot labels (1 for the correct class, 0 for all others), we use the teacher's output probability distribution (soft targets). For example, the teacher might output: 'cat=0.70, lynx=0.25, dog=0.05'. This distribution conveys much richer information about class relationships and similarities than a simple '1=cat'. The student learns these inter-class relationships alongside the correct classification.

EXAM QUESTION Q7 [~3 Points]

What is quantization in deep learning? Compare Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). What memory saving does INT8 provide over FP32?

MODEL ANSWER

Quantization reduces the precision of neural network weights and activations from high-precision floating point (FP32 = 32 bits) to lower-precision integer representations (e.g., INT8 = 8 bits). Since network weights rarely use the full FP32 range, this compression loses little information but dramatically reduces memory and compute costs.

Memory saving: FP32 uses 4 bytes per value. INT8 uses 1 byte per value. This provides a **factor-4 memory saving** — a model that required 4GB of memory now requires only 1GB.

PTQ (Post-Training Quantization): The model is trained normally in FP32, then the weights are quantized after training is complete. Simple to apply to any pre-trained model. Drawback: some accuracy loss because the model was never exposed to lower precision during training.

QAT (Quantization-Aware Training): Quantization is simulated during training — the model experiences the effects of lower precision while still having gradients to adapt. The model explicitly learns to compensate for quantization errors. Results in better accuracy than PTQ but requires re-training or a calibration training step.

EXAM QUESTION Q8 [~2 Points]

What is the difference between a GPU with CUDA and a TPU? Give two specific advantages of TPUs over GPUs for neural network training.

MODEL ANSWER

GPU + CUDA: A Graphics Processing Unit is a general-purpose parallel processing chip originally designed for rendering graphics, later adapted for deep learning. CUDA (Compute Unified Device Architecture) is NVIDIA's software platform that enables GPU use for scientific and ML computing. GPUs have thousands of cores that perform SIMD parallel operations.

TPU: A Tensor Processing Unit is a custom-designed chip by Google, built from the ground up specifically for neural network matrix operations. Unlike GPUs, every aspect of the TPU's design is optimised for ML workloads.

TPU advantage 1 — Higher Memory Bandwidth (HBM): TPUs have High Bandwidth Memory that feeds data to compute units much faster than typical GPU memory, reducing the data bottleneck.

TPU advantage 2 — Energy efficiency: TPUs deliver more computation per watt than GPUs, making large-scale training more cost-effective and environmentally sustainable.

QUICK REFERENCE CHEAT SHEET — EFFICIENT LEARNING

Optimisation Dynamics

Concept	Key Facts
Step Decay	Reduce LR by fixed factor every n epochs. Simple but abrupt.
Cosine Annealing	Smooth cosine decay. Popular for large training runs. Warm restarts to escape minima.
Cyclic (CLR)	Oscillates LR between min and max. Forces exploration.
Warm-Up	Start small, ramp up to target LR. Prevents early destabilisation.
Batch Norm	Normalise across mini-batch per feature. Learnable: beta (shift), gamma (scale). BN for CNNs.
Layer Norm	Normalise across all features per sample. Works at batch size=1. Best for RNNs/Transformers.

Generalisation, Hardware & Efficiency

Concept	Key Facts
L1 Regularisation	Loss += $\alpha \cdot \sum w $. Pushes weights to 0. Feature selection. Large α = strong penalty.
L2 Regularisation	Loss += $\alpha \cdot \sum (w^2)$. Keeps weights small but non-zero. Default regulariser.
Dropout	Randomly drop neurons at training. Scale outputs by (1-p) at inference. Prevents co-adaptation.
Early Stopping	Stop when validation loss stops improving. Free regularisation. Use patience parameter.
CUDA	NVIDIA GPU platform. Parallel SIMD matrix ops. TensorFlow/PyTorch use it automatically.
TPU	Google custom chip. Optimised for matrix ops. HBM (high bandwidth). 4x+ faster than GPU for DL.
Distillation	Teacher (large) → Student (small). Student learns soft labels. Compact+fast inference model.
PTQ	Post-Training Quantization: quantize after training. Simple. Some accuracy loss. FP32 → INT8 = 4x saving.
QAT	Quantization-Aware Training: simulate quantization during training. Better accuracy than PTQ.

L1 Loss = Error + $\alpha \cdot \text{SUM}|w|$ | L2 Loss = Error + $\alpha \cdot \text{SUM}(w^2)$ | FP32 → INT8 = 4x memory saving

EXAM STRATEGY — Efficient Learning: (1) For any normalisation question: state what dimension it normalises across and which models it suits. (2) For regularisation: always mention training error vs. test error trade-off. (3) For dropout: don't forget that inference behaviour differs (all neurons active, outputs scaled). (4) For distillation: always contrast teacher vs. student size and explain soft labels. (5) For quantization: state the memory saving factor (FP32 → INT8 = 4x) and PTQ vs QAT trade-off. (6) For hardware: CUDA = NVIDIA software platform; TPU = Google custom chip with HBM.